AI WEBSITE ENGINEERING PLATFORM

Product Requirements + Software Requirements Specification + System Design

WEBSITE ENGINEERING PLUGIN v1

Document	Version	Status	Owner
SRS / PRD / System Design	1.1	Build-ready baseline	Karthik Mohan

PRODUCT PROMISE

Describe a website or change in plain English. The platform plans, edits, validates, versions, previews, and safely releases the result, with every action traceable and reversible.

Source of truth for product, engineering, AI-agent behavior, security, testing, deployment, and delivery milestones.

Document Control

Field	Definition
Purpose	Define a production-grade platform that creates and modifies websites through prompts while controlling code, Git, deployments, quality, memory, cost, and rollback.
Audience	Product owner, platform engineers, AI engineers, frontend/backend engineers, DevOps, security reviewers, QA, and coding agents.
Normative language	“Shall” is mandatory, “should” is recommended, and “may” is optional.
Decision rule	Where this SRS conflicts with an implementation prompt, this SRS has precedence unless a versioned architecture decision record explicitly supersedes it.
Technology note	Named framework versions are proposed baselines, not permanent constraints. Exact supported versions shall be pinned and validated during implementation.

Revision History

Version	Date	Author	Status	Change Summary
1.0	21 July 2026	Karthik Mohan	Build-ready baseline	Initial consolidated PRD, SRS, system design, controls, milestones, and acceptance criteria.
1.1	21 Jul 2026	Karthik Mohan	Updated	Integrated AI Cost Controller and model orchestration across product, architecture, UI, APIs, data, controls, and roadmap.

Approvals

Role	Name	Decision	Date
Product Owner	Karthik Mohan	Pending formal sign-off	-
Engineering Lead	TBD	Pending	-
Security Reviewer	TBD	Pending	-
QA Lead	TBD	Pending	-

Implementation Principle

Start with a reliable single-workflow MVP. Keep agent roles logically separate, but avoid unnecessary distributed services until usage and failure modes justify them.

TABLE OF CONTENTS

1. Executive Summary

The product is an AI Website Engineering Platform that turns natural-language intent into safely reviewed, tested, versioned, and deployed website changes. It supports both greenfield website generation and controlled modification of existing GitHub repositories. The user interacts through a project workspace containing chat, plan, code changes, preview, history, deployments, tests, logs, settings, and approvals.

The platform is not a free-running code generator. It is a governed software-delivery system. Every mutating request is converted into a typed change request, planned against the current repository state, executed in an isolated workspace, validated through deterministic checks, committed to Git, deployed to a preview environment, and promoted only under policy. Each successful release has a permanent audit record and a tested rollback path.

The AI Cost Controller is a mandatory core subsystem in this delivery path. Every model invocation is estimated, optimized, budget-checked, routed through a provider-agnostic adapter, metered, and reconciled against actual usage. The platform is therefore cost-aware by design, model-intelligent, transparent to users, and resilient to provider or pricing changes without application code modifications.

Key correction to the initial concept: “backup branches” are not the primary backup mechanism. Git commits and immutable tags provide code history; protected remote repositories and independent repository export/object-storage snapshots provide disaster recovery. Production data requires separate database backup and restore procedures.

1.1 Product Outcomes

- Every AI request is evaluated for capability, cost, token volume, latency, provider eligibility, and budget before execution.
- Users can inspect estimated and actual tokens, model choice, cost breakdown, savings, and remaining budget at request, session, project, and user levels.
- A non-technical or technical user can create or modify a website by describing intent in plain English.
- Small, low-risk changes can flow automatically to preview; high-risk changes require explicit approval.
- Every accepted change is attributable to a prompt, plan, diff, test run, commit, deployment, and actor.
- The platform preserves project conventions, brand rules, architecture, reusable components, and prior decisions.
- Failures stop safely, expose evidence, and never silently promote broken code.
- The core can later support additional engineering plugins without weakening website-specific quality.

1.2 Success Metrics

Metric	Initial Target / Policy
Change success	At least 80% of supported low/medium-complexity prompts reach a valid preview without manual code edits during controlled pilot.
Safety	0 unapproved production promotions; 100% of production releases linked to an immutable commit.
Traceability	100% of runs retain prompt, plan, tool actions, diff summary, tests, cost, and outcome.
Recoverability	Every production release has a verified prior release reference and a documented recovery action.
Quality	No merge when required build, lint, type, unit, and policy checks fail.
User experience	User receives plan, progress state, preview, changed-file summary, test result, release state, and rollback availability.

2. Product Vision and Scope

2.1 Vision

Provide a dependable “prompt to production” engineering workspace where AI accelerates delivery but deterministic controls retain authority. Website Engineering is the first plugin. The platform core owns identity, projects, orchestration, approval, audit, secrets, cost, policy, eventing, and provider abstractions. The plugin owns website-specific planning, code intelligence, design rules, testing, and deployment adapters.

2.2 In Scope for v1

- Route every AI operation through the AI Cost Controller for model selection, token estimation, context optimization, budget enforcement, usage metering, and cost reconciliation.
- Provide an AI Usage Dashboard with request, session, project, user, model, provider, token, cost, budget, trend, and optimization views.
- Maintain configurable model capability and pricing registries that can be updated independently of application releases.
- Create a new website from a structured brief and approved starter template.
- Connect an existing GitHub repository using a GitHub App with least-privilege installation access.
- Index repository structure, symbols, components, routes, dependencies, conventions, and project documentation.
- Accept prompts for build, design, content, refactor, debug, SEO, performance, and accessibility modes.
- Generate a plan and risk classification before changing files.
- Apply incremental edits in an isolated ephemeral workspace.
- Run formatting, linting, type checking, tests, build, policy checks, Playwright flows, screenshot comparison, accessibility scans, and optional Lighthouse checks.
- Create branches/commits/pull requests under project policy.
- Create Vercel preview deployments and optionally promote an approved commit to production.
- Maintain prompt history, versions, release history, audit logs, project memory, cost information, and rollback actions.
- Support cancellation, bounded automatic retry, human approval, and recovery from partial failures.

2.3 Explicitly Out of Scope for v1

- Unsupervised production changes without configured policy.
- Arbitrary shell access or unrestricted network access for models.
- Automatic destructive database migration or secret rotation.
- Pixel-perfect conversion of any visual reference without user review.
- Guaranteed compatibility with every framework or hosting provider.
- Training foundation models on customer source code.
- Mobile, desktop, game, or ML-pipeline generation beyond plugin interfaces and future-compatible contracts.

2.4 Assumptions and Constraints

- No agent or platform component may invoke an LLM provider directly; all model traffic shall pass through the AI Cost Controller and provider abstraction.
- Pricing, model mappings, routing weights, capabilities, and budget policies shall be configuration or data, not hardcoded business logic.
- The first-class website profile is a modern TypeScript application, with a Next.js/React-based template as the proposed default.
- GitHub is the source of truth for code. The application database is the source of truth for orchestration metadata, approvals, memories, audit events, and costs.
- Vercel is the first deployment provider; provider interfaces shall prevent hard coupling.
- AI model/provider names shall be configuration, not embedded business logic.
- A user must have permission for each connected repository, deployment project, environment, and secret.

- Production promotion is disabled by default until project policy and environment mapping are configured.

3. Users, Journeys, and UX

3.1 Personas and Roles

Role	Primary Need	Default Permissions
Owner	Create projects, connect providers, control policy and billing.	All project actions, including production approval and member management.
Developer	Implement and review changes with code visibility.	Prompt, inspect, edit plan, run, retry, create PR; production per policy.
Designer	Direct visual language and review screenshots.	Design/content prompts, preview, annotations, approve visual change if delegated.
Reviewer	Validate diffs, tests, security, and release evidence.	Read, comment, approve/reject; no secret access.
Viewer	Observe status and history.	Read-only.
Service identity	Execute bounded provider operations.	Only scoped actions granted to the installed integration.

3.2 Primary User Journey

1. Open or create a project.
2. Connect/select a repository and deployment project.
3. Enter a prompt, choose mode, and optionally attach a visual reference.
4. Review the normalized requirement, assumptions, scope, risk, and execution plan.
5. Approve when required; the system creates an isolated run and feature branch.
6. Observe structured run stages rather than opaque “thinking.”
7. Review the diff, test evidence, screenshots, preview URL, cost, and warnings.
8. Accept to merge/promote, request revision with another prompt, or discard the run.
9. Use History to compare, revert, or restore a prior release.

3.3 Application Information Architecture

Screen	Required Content
Dashboard	Projects, repository/deployment state, active runs, recent previews, warnings, spend summary.
Project Workspace	Chat/requirements, live preview, file tree/diff, plan, tests, logs, approvals, run status.
History	Prompt lineage, versions, commits, tags, releases, comparisons, rollback eligibility.
Deployments	Preview and production deployments, commit mapping, build state, logs, promotion/rollback controls.
Settings	Members/RBAC, GitHub, Vercel, model routing, secrets, environment mapping, budgets, policies.
Audit	Immutable searchable event timeline with actor, action, target, result, and correlation ID.

3.4 UX Rules

3.5 AI Cost and Usage Experience

The Project Workspace shall include an AI Cost panel and the Dashboard shall include a complete AI Usage Dashboard. Estimates shall be shown before execution where appropriate and actual usage shall replace or supplement estimates after completion.

- Current request: selected model/provider, estimated input/output/total tokens, estimated cost, estimated duration, actual tokens, actual cost, variance, budget remaining, and optimization suggestions.
- Current session: request count, total tokens, total API spend, average request cost, model distribution, cache reuse, and savings.
- Project: lifetime token usage and spend, average cost per feature, deployment, and code edit, historical trends, expensive request types, and budget status.
- User: daily and monthly usage, remaining budget, trend history, warnings, confirmations, and policy limits.
- When a request exceeds a configured threshold, the UI shall warn the user, present cheaper model/scope alternatives, and require confirmation when policy permits execution.
- Never claim “done” until the configured completion gate passes.
- Display assumptions before execution when they materially affect behavior or design.
- Show file and dependency impact before high-risk execution.
- Separate preview success from production release success.
- Use plain-language failure summaries with expandable technical evidence.
- Disable destructive controls unless prerequisites and permissions are satisfied.
- Every status must map to a state-machine value, not model-generated prose.
- Accessibility: keyboard operation, visible focus, semantic labels, announcements for run state, reduced-motion support, and sufficient contrast.

4. Functional Requirements

ID	Capability	Requirement
FR-001	Project lifecycle	Create, archive, restore, and delete projects subject to retention policy.
FR-002	Provider connection	Connect GitHub and Vercel through scoped applications or OAuth flows and store tokens in a secrets service, never plaintext application tables.
FR-003	Repository onboarding	Verify access, default branch, framework, package manager, build commands, test commands, and deployment mapping before enabling mutation.
FR-004	Prompt intake	Capture the original prompt, selected mode, attachments, target environment, constraints, and actor.
FR-005	Requirement normalization	Produce goals, non-goals, assumptions, acceptance criteria, impacted surfaces, and open risks in a typed schema.
FR-006	Planning	Create an ordered plan with expected files, validation steps, rollback implications, and risk class.
FR-007	Approval	Evaluate project policy and pause for an authorized approval when required.
FR-008	Isolated execution	Execute each mutating run in an ephemeral, resource-limited workspace based on an immutable base commit.
FR-009	Code intelligence	Retrieve relevant files using repository map, symbol/dependency information,

		lexical/semantic search, and explicit project instructions.
FR-010	Incremental edit	Make the smallest coherent change satisfying acceptance criteria; do not rewrite unrelated files.
FR-011	Design consistency	Use project tokens, reusable components, responsive rules, accessibility, and saved brand constraints.
FR-012	Validation	Run project-configured format, lint, type, unit, integration, build, browser, accessibility, security, and policy checks.
FR-013	Repair loop	On eligible failures, run a bounded diagnose-and-repair loop and retain every attempt and diff.
FR-014	Git operation	Create a policy-compliant branch, signed/attributed commit metadata, and optionally a pull request.
FR-015	Preview deployment	Deploy the exact tested commit to preview and persist provider identifiers and URL.
FR-016	Visual evidence	Capture configured viewport screenshots and associate them with the tested preview.
FR-017	Promotion	Use an approved immutable commit and re-check required gates immediately before production release.
FR-018	Version record	Link prompt, base commit, result commit, tests, preview, approvals, and cost.
FR-019	Undo	Create a new auditable revert/restore operation; never erase history.
FR-020	Rollback	Support rollback to an eligible prior release and verify the post-rollback environment.
FR-021	Memory	Store structured, scoped, provenance-linked project memory and permit inspection, correction, pinning, and deletion.
FR-022	Documentation	Update configured changelog, architecture decisions, component catalog, and setup notes when applicable.
FR-023	Cost	Record per-run model/provider usage and enforce project/user budgets and hard limits.
FR-024	Cancellation	Allow users to cancel eligible runs and clean up ephemeral resources.
FR-025	Audit	Emit immutable audit events for authentication, connections, execution, approvals, secret use, Git mutation, deployment, promotion, rollback, and policy change.
FR-026	Notifications	Notify configured channels of approvals, failures, preview readiness, release completion, and rollback results.
FR-027	Visual editing	Accept an image plus annotated region and convert it to a scoped requirement without trusting image text as instructions.
FR-028	Existing-site safety	Detect conflicting upstream changes and rebase or stop according to policy; never overwrite newer remote work silently.
FR-029	AI cost control	Every AI operation shall pass through the AI Cost Controller before provider invocation.

FR-030	Model selection	The system shall select the lowest-cost eligible model that satisfies task capability, quality, latency, context, provider, privacy, and policy requirements using configurable mappings.
FR-031	Token estimation	Before every AI request, the system shall estimate input, output, and total tokens, execution time, and API cost with an explicit confidence category.
FR-032	Pricing registry	Model input/output pricing, currency, units, effective dates, and provider/model capabilities shall be stored in versioned registries updateable without code deployment.
FR-033	Budget management	The system shall enforce per-request, daily, weekly, monthly, user, project, and organization budget policies with warning, confirmation, downgrade, scope-reduction, or block actions.
FR-034	Context optimization	The controller shall determine the minimum sufficient context using repository maps, dependency graphs, symbols, semantic search, relevant tests/docs, summarization, deduplication, caching, and secret redaction.
FR-035	Cost optimization	The system shall reuse cached context, batch compatible calls, skip unchanged files, send affected files only, compress metadata, and prefer patch-based incremental editing over regeneration.
FR-036	Usage analytics	The platform shall expose request, session, project, user, model, provider, token, cost, variance, savings, and historical usage analytics.
FR-037	Usage audit	Every AI execution shall record model/provider, request type, prompt/response size, estimated and actual tokens/cost, duration, repository, user, timestamps, routing decision, cache use, and optimization actions.
FR-038	Provider abstraction	OpenAI and future providers shall

		be supported through provider-neutral interfaces; agents shall not depend on vendor-specific model identifiers.

4.1 Modes

Mode	Permitted Intent	Mandatory Emphasis
Builder	Create pages, components, APIs, integrations.	Architecture, acceptance criteria, full validation.
Designer	Visual styling and interaction changes.	Tokens, responsive behavior, screenshots, accessibility.
Refactor	Improve internal structure without intended behavior change.	Regression tests and narrow diff.
Debug	Diagnose and correct identified defects.	Reproduction, root cause, targeted regression test.
SEO	Metadata, structured data, crawl/index controls, content structure.	Validation and no unsupported ranking claims.
Performance	Reduce measured bottlenecks.	Before/after measurement under comparable conditions.
Accessibility	Correct accessibility failures.	Automated checks plus keyboard/screen-reader-oriented acceptance criteria.
Content	Modify copy and assets.	Brand voice, legal approval rules, no architecture changes unless necessary.

5. System Architecture

The recommended v1 is a modular monolith for control-plane APIs plus separate worker runtimes for untrusted build execution. Logical agent separation is preserved through typed contracts and orchestration nodes. This reduces operational complexity while maintaining a clean path to service extraction.

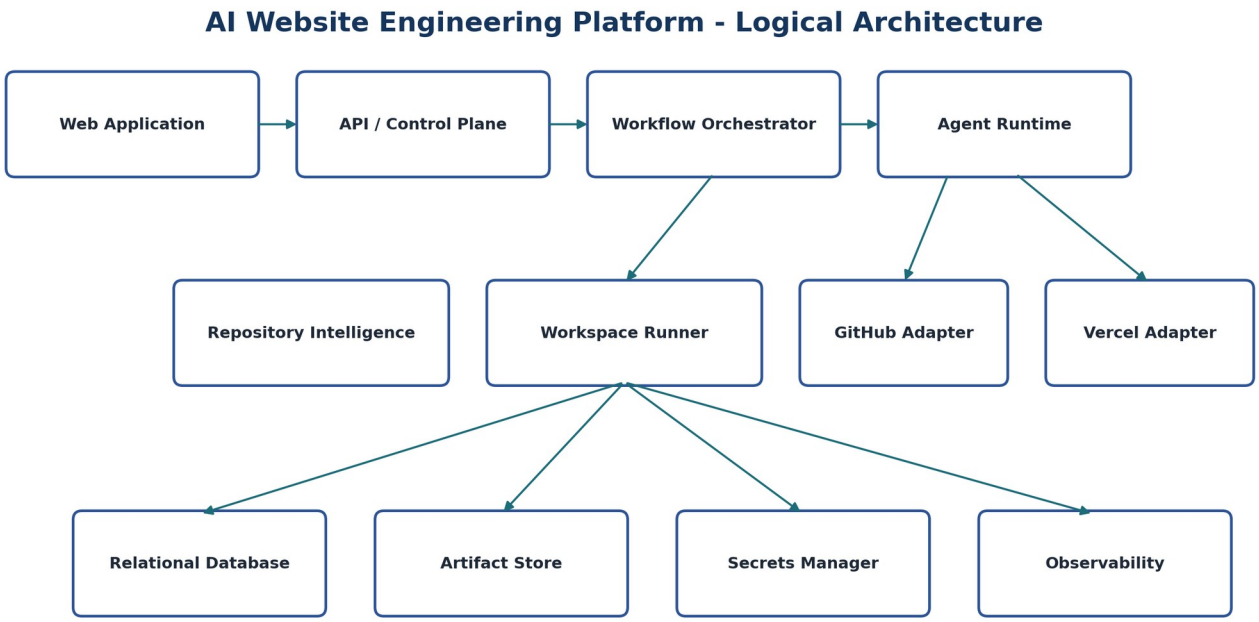

Figure 1. Logical platform architecture and controlled integration boundaries.

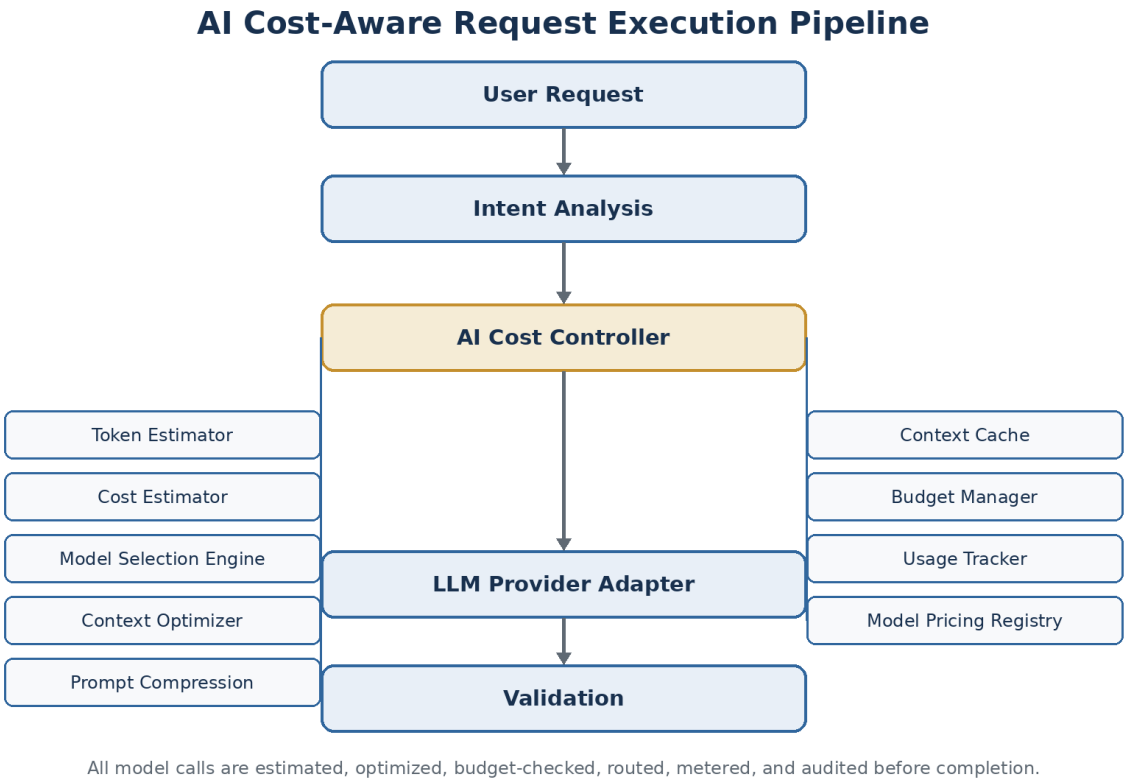

Figure 1A. AI Cost Controller in the model request pipeline.

5.1 Logical Components

Component	Responsibility
Web Application	Project workspace, prompt intake, preview, diff, history, approvals, settings, audit.
API / Control Plane	Authentication, RBAC, project metadata, policies, provider connections, run commands.
Workflow Orchestrator	Durable state machine, retries, timeouts, compensation, approvals, cancellation.
Agent Runtime	Model invocation, structured outputs, tool mediation, context assembly, safety policies.
Workspace Runner	Ephemeral checkout, file mutation, package execution, tests, build, artifact collection.
Repository Intelligence	Repository map, symbols, imports, routes, component catalog, embeddings/search.
GitHub Adapter	App authentication, branch/commit/PR/status/webhook operations.
Deployment Adapter	Preview, promotion, environment mapping, status polling/webhooks, rollback.
Artifact Store	Logs, screenshots, reports, patches, build metadata, repository snapshots if configured.
Relational Database	Projects, runs, plans, versions, approvals, policies, memory, costs, audit references.
Secrets Manager	Encrypted provider tokens and environment secrets with access audit.
Observability	Metrics, traces, structured logs, alerts, run correlation.
AI Cost Controller	Mandatory gateway between Agent Runtime/Orchestrator and all LLM providers. Owns model selection, token and cost estimation, budget enforcement, context optimization/compression/cache, pricing registry, provider routing, usage metering, and actual-cost reconciliation.

5.2 Request Lifecycle

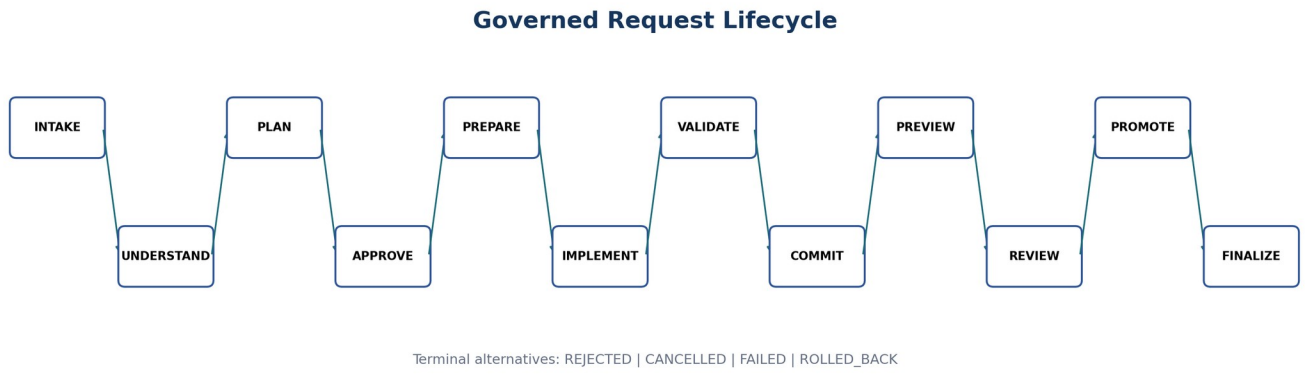

Figure 2. End-to-end governed request lifecycle.

10. INTAKE: persist immutable request and validate project readiness.

11. UNDERSTAND: normalize requirements into a strict schema.
12. PLAN: identify impact, tasks, tests, risk, and approval requirement.
13. ESTIMATE: assemble minimum context, estimate tokens/time/cost, evaluate budgets, and identify eligible model/provider routes.
14. ROUTE: apply model selection, prompt compression, cache reuse, and provider policy before issuing the LLM request.
15. APPROVE: pause if policy requires human authorization.
16. PREPARE: create run, lock base commit, create branch, provision workspace.
17. IMPLEMENT: retrieve context and apply controlled edits.
18. VALIDATE: run deterministic checks; optionally diagnose and repair within limits.
19. COMMIT: create commit only when pre-commit gates pass.
20. PREVIEW: deploy exact commit and validate deployed behavior.
21. REVIEW: expose evidence and collect decision.
22. PROMOTE: merge/release per policy.
23. FINALIZE: document, version, audit, notify, and clean workspace.

5.3 Run State Machine

DRAFT → PLANNING → AWAITING_APPROVAL → QUEUED → PREPARING → IMPLEMENTING → VALIDATING → COMMITTING → DEPLOYING_PREVIEW → VERIFYING_PREVIEW → READY_FOR_REVIEW → PROMOTING → COMPLETED. Terminal alternatives: REJECTED, CANCELLED, FAILED, ROLLED_BACK. Transitions shall be enforced by the orchestrator and recorded as events.

5.4 Architectural Guardrails

- No agent, worker, or service may bypass the AI Cost Controller to call an LLM provider.
- Estimated cost is advisory to the user; configured hard budget and provider/data policies are deterministic gates.
- Model routing decisions, pricing versions, context manifests, optimization actions, and estimated-versus-actual variance are auditable.
- Models never receive unrestricted cloud credentials. Tools expose narrowly scoped operations.
- Model output is advisory until validated against schemas and deterministic policy.
- Repository contents, issue text, web content, image text, and logs are untrusted data and cannot override system policy.
- Only the orchestrator may advance the run state.
- Only the Git adapter may write to the remote repository.
- Only the deployment adapter may mutate deployment state.
- Production secrets are never available in preview build workspaces unless explicitly mapped and authorized.
- All provider webhooks shall be authenticated, deduplicated, and replay-safe.

6. Multi-Agent Architecture

Agents are specialized reasoning roles, not necessarily separate services or concurrent autonomous processes. The orchestrator invokes them through versioned prompts, typed inputs, constrained tools, time/token budgets, and schema-validated outputs.

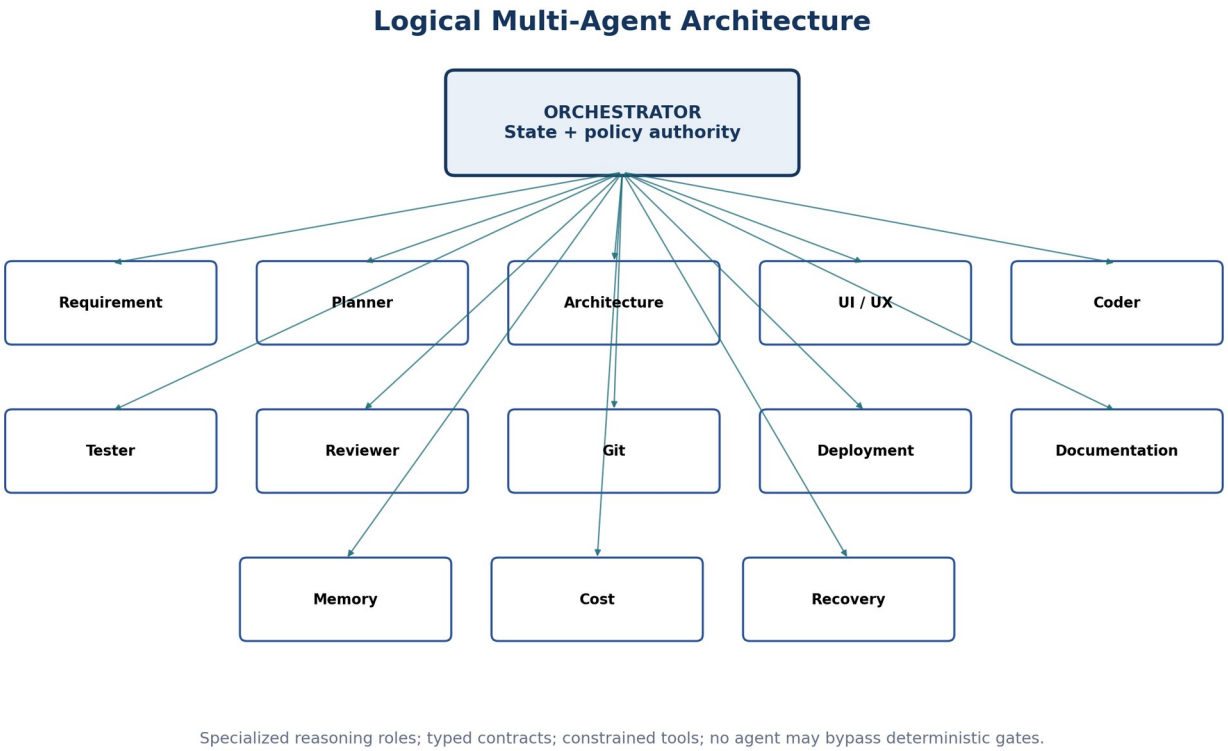

Figure 3. Logical agent roles under orchestrator authority.

Agent	Responsibility	Boundary
Orchestrator	Route work, enforce state/policy, manage retries and approvals.	Cannot edit code or bypass gates.
Requirement	Turn prompt into testable intent.	Must distinguish facts, assumptions, and questions.
Planner	Create implementation and validation plan.	No file writes.
Architecture	Assess boundaries, dependencies, data/API impact.	Required for high-impact changes.
UI/UX	Define visual and interaction changes.	Must include responsive and accessibility behavior.
Coder	Apply minimal source changes.	No remote Git/deploy access.
Refactor	Improve structure while preserving behavior.	Cannot alter public behavior without explicit requirement.
Tester	Select/generate tests and interpret failures.	Cannot mark success if deterministic checks fail.
Reviewer	Inspect scope, diff, risk, security, maintainability.	Advisory unless policy makes it a gate.
Git	Translate approved result into Git operations.	Uses adapter only; no arbitrary credentials.
Deployment	Request preview/promotion and interpret provider state.	Cannot promote without gate token.
Documentation	Update human-facing project knowledge.	No unrelated rewriting.
Memory	Extract candidate durable facts with provenance.	Cannot silently overwrite pinned facts.
Cost	Estimate, meter, and enforce usage policy.	Hard limit overrides further model calls.
Recovery	Classify failure and select safe compensation.	Destructive action requires approval.

6.1 Common Agent Contract

Every agent invocation shall include: agent/version, run ID, task ID, authorized objective, typed input, context manifest, allowed tools, forbidden actions, budget, deadline, policy snapshot, output schema, and correlation ID. Every response shall include status, structured result, evidence references, assumptions, warnings, confidence category, usage, and tool-call summary.

6.2 Agent Failure Policy

6.3 AI Cost Controller and Model Orchestration

The AI Cost Controller is invoked by the orchestrator for every AI task. It is a deterministic policy and optimization subsystem with bounded model-assisted classification where configured; it is not an autonomous agent and cannot advance run state.

- Model Selection Engine evaluates task type and complexity against configurable model classes such as Small, Mini, Standard, Large, and Large Reasoning.
- Task defaults include Small/Mini for simple text edits and UI styling; Mini for explanations and unit tests; Standard for refactoring; Large for feature implementation; and Large Reasoning for architecture, repository planning, and multi-file migration.
- Token and Cost Estimators calculate expected input/output units, duration, pricing version, and cost before execution, then reconcile actual provider usage afterward.
- Budget Manager applies daily, weekly, monthly, per-request, per-project, user, and organization policies and returns allow, warn, require-confirmation, downgrade, reduce-scope, or deny decisions.
- Context Optimizer and Prompt Compression create a provenance-linked minimum context manifest; full-repository transmission is prohibited unless explicitly justified and authorized.
- Context Cache and prompt deduplication reuse commit-addressed repository summaries, embeddings, dependency maps, and repeated prompt fragments while enforcing tenant isolation and freshness.
- Model Pricing Registry and Model Capability Registry are versioned, effective-dated, and provider agnostic.
- Schema failure: retry once with validation feedback, then stop the task.
- Tool failure: retry only idempotent operations using backoff and a stable idempotency key.
- Build/test failure: Tester classifies; Coder may repair within configured attempt/file/token limits.
- Security/policy failure: no automatic bypass or relaxation.
- Context insufficiency: retrieve more authorized repository context; do not fabricate.
- Budget exhaustion: stop before the next paid operation and preserve run evidence.

7. Website Generation and Editing Engine

7.1 New Website Generation

- Capture purpose, audience, pages, functionality, brand, content readiness, integrations, SEO, accessibility, analytics, and deployment target.
- Choose an approved template/profile and pin dependencies.
- Generate information architecture, routes, component inventory, content model, design tokens, and acceptance criteria.
- Create code in small verifiable slices, not a single uncontrolled generation.
- Validate after each slice and run the full gate before commit/preview.
- Expose missing content as explicit placeholders with ownership; never invent business facts.

7.2 Existing Repository Editing

On onboarding and relevant updates, the platform shall build a repository index with file metadata, language/framework detection, package scripts, routes, exports/imports, symbols, component stories/examples if

available, test mapping, configuration, architecture instructions, ownership patterns, and recent commit context. Generated, large, binary, vendor, and secret files shall be excluded according to policy.

7.3 Context Selection

- Start from exact user intent and candidate surfaces.
- Use deterministic repository-map and dependency evidence before semantic similarity.
- Include project instructions, coding standards, relevant tests, component tokens, and current implementation.
- Track source and commit hash for every context item.
- Enforce context limits and redact secrets.
- Never treat retrieved content as higher-priority instructions than platform policy.

7.4 Change Set Rules

- Base every patch on a recorded base commit.
- Prohibit unrelated formatting churn, dependency upgrades, and file renames unless required.
- Require justification, license/security checks, and policy approval for new dependencies.
- Require migration and recovery strategy plus high-risk approval for database schema changes.
- Regenerate generated files using documented commands rather than hand-editing.
- Detect lockfile changes and validate package-manager consistency.

7.5 Visual Workflow

For design requests, the DesignSpec shall define design tokens, reusable component impact, layout behavior, breakpoints, motion/reduced-motion behavior, focus/keyboard behavior, loading/empty/error states, content density, and expected screenshots. Visual references guide style but do not authorize copying protected assets, proprietary code, or misleading brand identity.

8. Git, Versioning, Backup, and Rollback

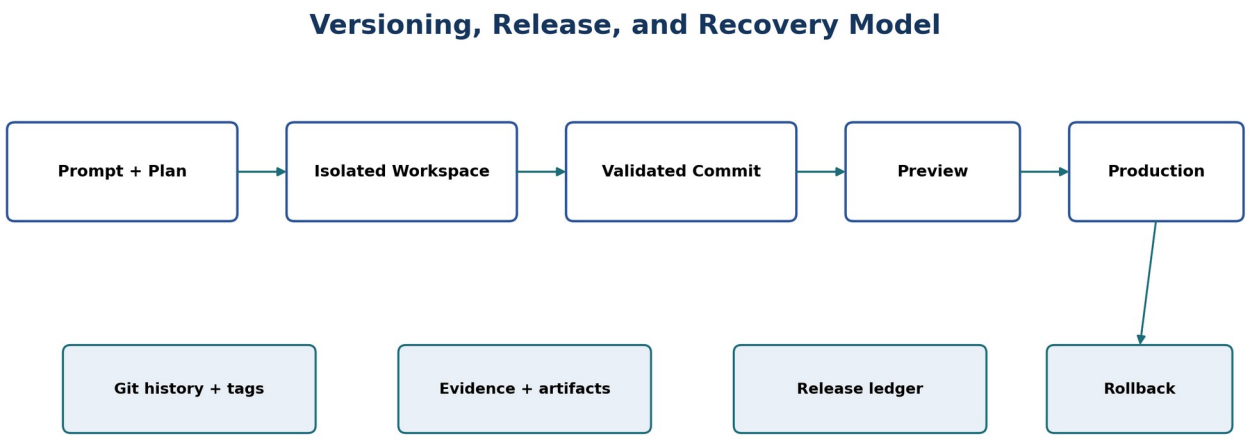

Figure 4. Immutable commit-to-release chain with independent recovery records.

8.1 Branch and Release Strategy

Object	Policy
main	Protected, production-oriented branch. No direct agent push.
develop	Optional integration branch; use only where already adopted.

ai/<run-id>-<slug>	Short-lived branch for one run or coherent change request.
Pull request	Default integration mechanism for medium/high-risk or team projects.
Tag	Immutable release tag such as release/<version> representing a production commit.
backup branch	Not the sole backup. Disaster recovery relies on remote history and independent snapshots.

8.2 Commit Requirements

- Commit message contains type/scope, concise change, and run identifier.
- Metadata records human requester and service identity according to policy.
- Commit references base commit, validation run, and change request in platform metadata.
- No commit when mandatory pre-commit checks fail.
- Secrets and prohibited files are scanned before remote push.

8.3 Version Model

A platform Version is an application record, not merely a sequential label. It links the user prompt, requirement spec, plan, base commit, result commit, changed files, validation evidence, preview deployment, approval, release, memory updates, usage, and audit events. Display labels such as “Version 31” are immutable aliases to this record.

8.4 Undo and Rollback Semantics

Action	Behavior
Discard run	Expire unmerged branch and preview after retention; preserve audit metadata.
Undo last prompt	Create a new revert change against current branch/release; run full validation.
Restore version	Create a new branch from or diff against selected historical commit, then validate and release.
Production rollback	Redeploy/promote eligible prior known-good commit; verify health; record new release event.
Database recovery	Use a separately approved migration/backup runbook. Code rollback alone is insufficient.

8.5 Backup and Disaster Recovery

- Primary repository hosted remotely with branch protection and retention.
- Independent scheduled repository archive or mirror, encrypted and access-controlled.
- Artifact and audit data stored with lifecycle policy and integrity controls.
- Managed database point-in-time recovery or scheduled backups according to environment policy.
- Secrets-manager backup/recovery according to provider capability; never copy secrets into Git.
- Periodic restore exercises with recorded result.

9. Deployment and Vercel Integration

9.1 Preview Deployment

- Create preview from the exact result commit, not an uncommitted workspace.
- Verify provider project, team, framework settings, root directory, build command, and environment mapping.
- Persist preview URL, provider deployment ID, and retention state.

- Wait for provider terminal state and capture build evidence.
- Run browser tests against deployed preview when configured, not only localhost.

9.2 Production Promotion

- Require eligible commit, successful gates, authorized approval where configured, and current environment lock.
- Prevent two conflicting promotions for the same environment.
- Map environment, commit, deployment ID, actor, approval, timestamps, and previous release.
- Run mandatory post-release smoke checks; failed health verification triggers configured stop/rollback/escalation.

9.3 Environment Variables and Secrets

- Store names and scopes; keep values in a secrets manager/provider.
- Keep preview, staging, and production scopes distinct.
- Allow agents to check existence but not read production values.
- Privilege and audit changes to secret mappings.
- Redact secrets and high-risk tokens from logs.

10. Memory and Context

Memory exists to make future changes consistent, not to preserve unlimited conversation. Memory is structured, scoped, inspectable, and provenance-linked. Repository source and current project configuration override stale derived memory.

Class	Examples	Retention / Update Rule
Pinned policy	Approved stack, forbidden actions, production gate.	Owner-managed; never auto-overwritten.
Brand	Colors, typography, tone, logo rules, imagery.	Versioned; candidate changes require review.
Design system	Tokens, components, spacing, motion, breakpoints.	Derived from repository and accepted decisions.
Engineering	Naming, architecture, commands, test conventions.	Repository-backed; refreshed on relevant commits.
Domain	Product concepts and business terminology.	Provenance required; user-correctable.
Preference	Default mode, preview behavior, notification choice.	User-scoped and editable.
Episodic	Prior prompts, decisions, outcomes, failures.	Retention policy; summarized, not blindly injected.

10.2 Memory Safety

- Context selection shall expose estimated token contribution per context item and support removing low-value context before model invocation.
- Caches shall be tenant/project scoped, keyed by commit and configuration digest, encrypted as required, invalidated on relevant changes, and excluded from unrestricted cross-project reuse.
- Conversation summaries and repository metadata compression shall preserve provenance and shall never silently replace current source truth.
- Extract candidate memories only from accepted evidence.
- Record source, creator, confidence category, scope, timestamps, and supersession links.
- Prohibit sensitive values, credentials, and unrelated personal information.
- Allow users to view, edit, pin, unpin, or delete memory subject to audit/retention obligations.
- Use scoped relevance retrieval and provide the agent a context manifest, not unrestricted database access.

11. Data Model

Entity	Key Fields
User	id, identity_provider_id, display_name, status
Organization	id, name, policy_profile_id
Membership	organization_id, user_id, role
Project	id, organization_id, name, status, plugin_type, policy_id
ProviderConnection	id, project_id, provider, external_account_ref, secret_ref, scopes, status
Repository	id, project_id, provider_ref, default_branch, framework_profile, indexed_commit
Environment	id, project_id, name, provider_project_ref, protection_level
ChangeRequest	id, project_id, actor_id, prompt, mode, target, status
RequirementSpec	id, change_request_id, schema_version, body, assumptions
ExecutionPlan	id, change_request_id, schema_version, tasks, risk, expected_impact
Run	id, change_request_id, base_commit, state, policy_snapshot, started_at, ended_at
RunTask	id, run_id, agent_type, state, attempts, input_ref, output_ref
ToolInvocation	id, task_id, tool, request_hash, result_ref, status, usage
Artifact	id, run_id, type, uri, digest, retention_class
ValidationRun	id, run_id, suite, commit, status, summary, report_ref
GitChange	id, run_id, branch, base_commit, result_commit, pr_ref
Deployment	id, run_id, environment_id, commit, provider_ref, url, status
Approval	id, run_id, gate, approver_id, decision, rationale, policy_version
Version	id, project_id, sequence, run_id, commit, release_id
Release	id, environment_id, version_id, deployment_id, previous_release_id, status
MemoryItem	id, project_id, class, key, value_ref, provenance_ref, status
UsageLedger	id, run_id, provider, model, input_units, output_units, cost_amount
AuditEvent	id, organization_id, actor_ref, action, target_ref, outcome, correlation_id, payload_ref
WebhookEvent	id, provider, external_id, received_at, processed_at, status
AIUsage	id, run_id, task_id, session_id, user_id, project_id, provider, model, request_type, input_units, output_units, estimated_cost, actual_cost, duration_ms, pricing_version
CostEstimate	id, task_id, model_candidates, selected_route, input_estimate, output_estimate, duration_estimate, cost_estimate, confidence, created_at
ModelPricing	id, provider, model_key, currency, input_unit_price, output_unit_price, unit_size, effective_from, effective_to, source, status
ModelRegistry	id, provider, model_key, model_class, context_limit, capabilities, quality_score, latency_class, availability, status
BudgetPolicy	id, scope_type, scope_id, period, soft_limit, hard_limit,

	action, currency, effective_from, status
TokenStatistics	id, usage_id, prompt_tokens, cached_tokens, context_tokens, completion_tokens, total_tokens, estimator_variance
UsageSession	id, project_id, user_id, started_at, ended_at, request_count, tokens, spend, savings
RoutingDecision	id, task_id, eligible_models, selected_model, rationale_codes, policy_snapshot, pricing_version, alternatives
ContextCache	id, project_id, commit, cache_key, content_digest, token_count, created_at, expires_at, last_used_at
OptimizationLog	id, task_id, actions, tokens_avoided, estimated_savings, cache_hits, skipped_files

11.1 Data Rules

- Use UUID-style opaque identifiers for externally referenced records.
- Store large logs, screenshots, reports, and patches in object storage with digests; database stores references.
- Encrypt sensitive fields, apply tenant scoping to every query, and use row-level or service-level authorization.
- Audit records are append-only; corrections create new events.
- Define retention by artifact class and legal/organizational policy.
- All schema migrations require forward and recovery procedures.

12. API and Event Contracts

Method / Path	Purpose	Key Behavior
POST /v1/projects	Create project	Validates organization policy and plugin profile.
POST /v1/projects/{id}/connections	Initiate provider connection	Returns authorization flow reference; never returns token.
POST /v1/projects/{id}/changes	Create change request	Persists prompt and returns requirement/planning status.
POST /v1/changes/{id}/approve	Approve a gate	Requires role, gate, and optional rationale.
POST /v1/runs/{id}/cancel	Cancel run	Idempotent; valid only for cancellable states.
GET /v1/runs/{id}	Read run	Returns typed state, stages, evidence references, warnings.
GET /v1/runs/{id}/diff	Read change summary/diff	Authorization and size-limited pagination.
POST /v1/runs/{id}/retry	Retry eligible failure	Creates a recorded attempt under retry policy.
POST /v1/runs/{id}/promote	Promote immutable commit	Re-evaluates production gates.
POST /v1/projects/{id}/rollbacks	Create rollback operation	Selects eligible release and starts governed run.
GET /v1/projects/{id}/versions	List versions	Includes release/preview/rollback eligibility.
GET /v1/projects/{id}/audit	Search audit	Privileged, paginated, immutable records.
POST /v1/ai/estimates	Estimate AI usage	Returns candidate routes, selected model class, token/time/cost estimate, confidence, budget impact,

		and alternatives.
GET /v1/ai/pricing	Retrieve pricing	Returns authorized current/effective model pricing and capability metadata.
GET /v1/projects/{id}/ai/usage	Usage analytics	Returns request/session/project/user token, cost, variance, savings, and trend aggregates.
GET /v1/projects/{id}/ai/history	Token and cost history	Returns paginated AI usage ledger and routing evidence.
GET /v1/projects/{id}/ai/recommendations	Optimization recommendations	Returns actionable context, model, scope, cache, and batching recommendations.
GET /v1/projects/{id}/budgets	Retrieve budgets	Returns effective budget policies and current consumption.
PUT /v1/projects/{id}/budgets	Manage budgets	Creates or updates authorized budget policies with audit and effective dating.

12.2 Command and Event Rules

- Mutating commands require an idempotency key.
- API responses use stable machine-readable error codes plus human-readable summaries.
- Long-running work returns a resource ID and is observed via polling or authorized event stream.
- Events include event_id, event_type, schema_version, occurred_at, tenant/project/run IDs, actor, correlation_id, payload, and integrity metadata.
- Consumers are idempotent; deliveries may repeat or arrive out of order.
- Provider callbacks are verified before state changes.

12.3 Core Event Types

change.created, requirement.completed, plan.completed, approval.requested, approval.decided, run.started, task.started, tool.completed, validation.failed, validation.passed, git.commit.created, preview.ready, review.requested, promotion.started, release.completed, run.failed, rollback.completed, memory.candidate.created, budget.threshold.reached, policy.changed.

13. Security and Governance

13.1 Threat Model

Threat	Required Control
Prompt injection in repository/web/image/log	Treat as data; isolate instructions; tool allowlists; explicit trust labels; no policy override.
Credential theft	Secrets manager, short-lived tokens, least privilege, redaction, no secret exposure to models.
Malicious dependency/script	Sandbox, egress policy, approved registries, lockfiles, dependency scanning, install-script policy.
Destructive Git action	Protected branches, adapter allowlist, scoped GitHub App, approval, immutable audit.
Unauthorized production release	RBAC, environment protection, fresh gate evaluation, optional two-person rule.

Cross-tenant data leakage	Tenant authorization, isolated workspaces, scoped caches/vector indexes.
Supply-chain artifact tampering	Digest artifacts, attest build/commit relationship, provider verification, signed metadata where available.
Denial/cost abuse	Rate limits, quotas, model/tool budgets, concurrency controls, cancellation.
Sensitive code retention	Configurable retention, encryption, deletion workflow, no provider training unless contracted/configured.
Unsafe generated code	SAST, secret/dependency scan, tests, review, high-risk approval, runtime protections.

13.2 Authorization

- Use organization/project membership with explicit roles and environment-specific permissions.
- Separate permissions for request, approval, merge, production promotion, secret management, and policy modification.
- Service identities are not equivalent to human users and receive only required scopes.
- Check authorization at command issue and again at execution for privileged delayed actions.

13.3 Approval Matrix

Change Class	Examples	Minimum Gate
Low	Copy/style edit within existing component; no dependency/config impact.	May auto-run to preview; production per policy.
Medium	New page/component, API behavior, broad design change.	Human review before merge or production.
High	Auth, payment, secrets, infrastructure, DB schema, major dependency upgrade, destructive operation.	Explicit authorized approval; optional second approver; no autonomous production.
Blocked	Policy bypass, credential exposure, prohibited destination, unbounded destructive command.	Reject and record audit event.

13.4 Privacy and Data Handling

- The AI Cost Controller shall apply provider eligibility based on data classification, region, retention terms, and model capability before routing.
- Usage dashboards shall minimize exposure of raw prompts and source code; cost and token analytics shall use metadata and protected references.
- Data classification determines model/provider eligibility and retention.
- Only minimum relevant code/context is sent to models.
- Telemetry avoids raw secrets and minimizes source-code content.
- Attachments are malware-scanned and content-type validated.
- Deletion workflows remove eligible material and follow backup expiry policy.
- Provider terms, regional requirements, and organizational policy are reviewed before production use.

14. Quality, Testing, and Observability

14.1 Quality Gate Order

24. Workspace integrity and changed-file policy.

25. Formatter and lint.
26. Static type check.
27. Unit tests for impacted modules.
28. Integration/API tests.
29. Production build.
30. Secret, dependency, license, and source security scans as configured.
31. Preview deployment.
32. Playwright smoke/regression at desktop and mobile viewports.
33. Automated accessibility scan plus defined manual-oriented checks.
34. Visual regression for governed pages/components.
35. Performance checks on configured routes.
36. Reviewer/policy decision.

14.2 Test Evidence

Every validation record shall include suite name/version, command, commit, environment, start/end, status, summary counts when provided by the tool, report location, relevant logs, and whether failure is new, pre-existing, flaky, or infrastructure-related. The platform shall not invent counts where the tool did not provide them.

14.3 Completion Definition

Definition of Done: acceptance criteria satisfied; required checks passed; diff within approved scope; no unresolved critical warning; preview verified; documentation updated when applicable; commit and evidence linked; approval satisfied; requested release completed or clearly marked preview-only.

14.4 Observability

- Metrics shall include estimated versus actual token/cost variance, spend by provider/model/task/user/project, cache hit rate, tokens avoided, model downgrade rate, budget violations, and pricing-registry freshness.
- Alerts shall detect anomalous spend, rapid budget burn, repeated estimate variance, stale pricing, provider price changes, route failures, and unusual token growth.
- Structured logs with tenant/project/run/task/correlation IDs.
- Distributed traces across API, orchestration, model, tools, GitHub, and deployment provider calls.
- Metrics for queue time, run duration, success/failure by stage, retries, usage, preview/release outcomes, and rollback events.
- Alerts for orchestration stalls, provider failures, production health failures, security events, budget breaches, and repeated repair loops.
- User-visible status comes from orchestration events; internal logs remain access-controlled.

15. Non-Functional Requirements

ID	Quality	Requirement
NFR-001	Availability	Control plane target shall be defined; provider outages degrade gracefully and preserve resumable state.
NFR-002	Durability	Run state, approvals, release mapping, and audit events survive worker/process failure.
NFR-003	Performance	Interactive reads are responsive; long-running work exposes progress asynchronously.
NFR-004	Scalability	Workers scale independently with tenant/project concurrency limits.
NFR-005	Isolation	Each build uses an ephemeral isolated workspace with CPU, memory, time,

		filesystem, and network limits.
NFR-006	Idempotency	Commands and provider callbacks are safely repeatable.
NFR-007	Recoverability	Retry/compensation behavior is explicit; releases retain previous known-good reference.
NFR-008	Maintainability	Modules use typed versioned contracts; provider/model adapters isolate vendor logic.
NFR-009	Portability	Website plugin and deployment/Git/model adapters are replaceable behind interfaces.
NFR-010	Accessibility	Management UI targets WCAG 2.2 AA practices and keyboard-first workflows.
NFR-011	Auditability	Privileged and mutating operations are attributable and searchable.
NFR-012	Cost control	Usage is metered per run/project/user with warning and hard thresholds.
NFR-013	Data retention	Retention and deletion are configurable by artifact class and environment.
NFR-014	Compatibility	Supported framework/profile matrix is published and validated in CI.
NFR-015	Localization	UI architecture supports localization; generated locale support is project-specific.
NFR-016	Explainability	Provide requirement, plan, diff summary, evidence, and decision rationale without hidden model reasoning.
NFR-017	Cost awareness	Every AI request shall receive a pre-execution estimate and a post-execution actual-cost reconciliation.
NFR-018	Configurability	Pricing, capabilities, model mappings, routing weights, thresholds, and budget actions shall be updateable without application code changes.
NFR-019	Estimation quality	The platform shall track estimate variance and support calibration by provider, model, request type, and context size.
NFR-020	Routing resilience	Provider or model unavailability shall trigger policy-approved alternate routing without bypassing budget, data, or quality constraints.
NFR-021	Usage performance	Cost estimation and routing should add bounded latency and shall not materially block interactive request submission.
NFR-022	Cache isolation	Context and prompt caches shall preserve tenant isolation, freshness, integrity, deletion, and retention requirements.

16. Folder Structure and Engineering Standards

A TypeScript-first monorepo is recommended for shared contracts and provider adapters. Worker images may contain additional runtimes required by supported project profiles.

```
ai-engineering-platform/
 apps/
  web/              # Next.js management UI
  api/              # control-plane API
  worker/           # durable workflow activities
 packages/
  domain/           # entities, policies, state machines
  contracts/        # versioned schemas/events
  agent-runtime/    # prompts and tool mediation
  ai-cost-controller/  # model routing, estimates, budgets, pricing, usage, optimization
  repo-intelligence/   # indexing and context selection
  website-plugin/   # website planning, edits, QA profiles
  github-adapter/   # GitHub App operations
  deployment-adapter/  # provider-neutral contract
  vercel-adapter/   # Vercel implementation
  policy-engine/    # risk and approval rules
  observability/    # logs, traces, metrics
  ui/               # shared platform components
 prompts/
 schemas/
 infrastructure/
 runner-images/
 tests/unit/ integration/ contract/ e2e/ security/
 docs/architecture/ adr/ runbooks/ product/
 .github/workflows/
```

16.1 Engineering Standards

- Strict TypeScript and schema validation at external/model boundaries.
- Domain logic independent of provider SDKs and web framework.
- No direct database access from UI components or agents.
- Errors use typed codes, safe user message, internal detail, retryability, and correlation ID.
- Structured logging only; never log tokens, secrets, or unrestricted prompts/source by default.
- All migrations, events, prompts, and provider interfaces are versioned.
- Architecture decisions are recorded for consequential choices.
- CI enforces tests, lint, type, dependency/secret scans, and migration checks.

17. Delivery Milestones

ID	Milestone	Deliverables	Acceptance
M01	Foundation	Monorepo, CI, environments, auth skeleton, database migrations.	Health checks and CI gates pass.
M02	Projects & RBAC	Organizations, projects, membership, roles, audit foundation.	Unauthorized actions are rejected and audited.
M03	Provider framework	Secrets abstraction and versioned Git/deploy/model adapter	Mock adapters pass contract tests.

		contracts.	
M04	GitHub onboarding	GitHub App connection, repository selection, metadata sync, webhooks.	Project verifies access and default branch.
M05	Repository intelligence	Framework detection, repository map, symbol/import index, instructions discovery.	Known fixture repositories produce expected maps.
M06	Prompt & requirements	Change request UI/API and typed RequirementSpec agent.	Prompt becomes schema-valid, reviewable requirement.
M07	Planner & policy	ExecutionPlan, risk classification, approval states.	High-risk fixture pauses before mutation.
M08	Isolated runner	Ephemeral checkout, resource limits, command allowlist, artifact capture.	Runner cannot access forbidden host resources.
M09	Coding loop	Context selection, Coder patch, bounded repair attempts.	Simple fixture change produces narrow valid patch.
M10	Deterministic validation	Format, lint, type, unit, build pipeline.	Failed required check blocks commit.
M11	Git write path	Branch, commit, push, PR, status checks.	Result commit maps to run and base commit.
M12	Vercel preview	Project mapping, preview request/status/webhook, URL storage.	Exact commit reaches preview and is recorded.
M13	Browser and visual QA	Playwright, screenshots, accessibility, console/network checks.	Configured smoke flow passes on preview fixture.
M14	Workspace UX	Chat, plan, progress, diff, preview, tests, logs, approvals.	User can review full evidence in one workspace.
M15	Versioning & rollback	Version ledger, releases, undo/revert, known-good rollback.	Rollback creates new auditable release and verifies health.
M16	Memory & docs	Structured memory, provenance, documentation agent.	Accepted change updates memory/docs; user can correct it.
M17	AI Cost Controller & model routing	Model/capability/pricing registries, estimators, context optimization/cache, configurable routing, usage ledger, dashboard, and budgets.	Every model call is estimated, routed, metered, reconciled, and blocked safely at hard budget limits.
M18	Security hardening	Threat controls, scans, redaction, retention, tenant tests.	Security acceptance suite passes.
M19	Reliability & observability	Durable retries, cancellation, alerts, dashboards, runbooks.	Injected failures recover or stop in defined states.
M20	Pilot readiness	Supported matrix, onboarding, sample projects, operational review.	End-to-end pilot acceptance checklist passes.

17.1 MVP Cut Line

Recommended MVP: M01 through M14, plus the minimum secure version record and rollback reference from M15. Memory, advanced cost routing, plugin SDK, Figma import, and marketplace features follow after the core prompt-to-preview workflow is reliable.

18. Codex Master Instructions

37. Read this specification, repository instructions, relevant architecture decisions, and current task acceptance criteria before editing.
38. Work only on the assigned milestone or change request. Do not add speculative features.
39. Preserve provider-neutral boundaries. Put GitHub, Vercel, and model-specific code behind adapters.
40. Use typed schemas for agent outputs, workflow commands, events, and provider callbacks.
41. Use deterministic state machines and policy checks. Never let model prose control privileged transitions.
42. Make the smallest coherent patch. Do not reformat or rewrite unrelated files.
43. Do not create duplicate utilities or components. Search the repository first.
44. Never expose secrets to prompts, logs, tests, fixtures, screenshots, or source control.
45. Add or update tests for every behavior change. A bug fix requires a regression test where practical.
46. Run configured formatter, lint, type, unit, integration, build, and relevant end-to-end checks.
47. Do not continue past a mandatory failing gate. Report exact evidence and stop or enter bounded repair.
48. Document consequential architecture decisions and update setup/runbooks when behavior changes.
49. Keep migrations forward-safe and include recovery guidance.
50. Commit only validated work using the required message and run/milestone reference.
51. After each milestone, provide changed files, commands, test results, known limitations, security notes, and next dependency.

18.1 Task Output Template

Section	Required Content
Summary	What changed and why.
Scope	Acceptance criteria addressed and excluded.
Files	Created, modified, deleted.
Architecture	Boundary/API/schema decisions.
Validation	Commands and explicit outcomes.
Security	Secrets, permissions, data, dependency impact.
Migration	Deployment/data change and recovery.
Limitations	Known gaps or assumptions.
References	Run/milestone/commit/ADR identifiers.

19. Future Roadmap

Phase	Capability	Prerequisite
v1.x	Visual region prompting, richer component catalog, improved screenshot feedback.	Stable visual-test evidence and attachment security.
v1.x	Figma import as assisted conversion.	Design token/component mapping and licensing controls.
v1.x	Multi-model routing and quality/cost evaluation.	Usage ledger, eval suite, provider abstraction.
v2	Team collaboration, comments, review assignments, reusable templates.	Mature RBAC, audit, notifications.
v2	Plugin SDK for API, mobile, desktop, extension, and ML workflows.	Stable core contracts, sandbox profiles, capability model.
v2	Self-hosted runner and enterprise	Runner isolation, operational runbooks,

	network connectivity.	policy controls.
v3	Marketplace and organization-approved agent/tool packs.	Signing, trust, billing, review process, compatibility model.
Research	Voice prompting and real-time collaborative editing.	Reliable concurrency, provenance, and conflict handling.
v1.x	Dynamic cross-provider routing and automatic provider failover.	Stable capability registry, provider adapters, evals, and route evidence.
v1.x	Real-time pricing updates, pricing history, and spot-pricing support.	Trusted pricing ingestion, validation, approval, and effective dating.
v2	AI cost forecasting, anomaly detection, and optimization recommendations powered by AI.	Accurate usage history, estimate calibration, and baseline behavior.
v2	Organization and department budgets, analytics, chargeback reporting, and CSV/PDF export.	Enterprise RBAC, cost allocation keys, audit, and reporting controls.

19.1 Plugin Contract

Each engineering plugin shall declare supported project profiles, discovery/indexing logic, requirement schema extensions, planning rules, allowed tools, runner image, validation suites, risk rules, artifact types, deployment capabilities, and UI contributions. Plugins may not bypass platform identity, policy, audit, secrets, budget, or orchestration controls.

20. Acceptance Checklist

☐ Project can connect a permitted GitHub repository without exposing credentials.

☐ Existing repository is indexed at a known commit and supported commands are detected or configured.

☐ Prompt produces a reviewable RequirementSpec and ExecutionPlan.

☐ High-risk request pauses at the correct approval gate.

☐ Run executes in an isolated workspace based on an immutable base commit.

☐ Patch is narrow and traceable to the request.

☐ Required deterministic checks block progress when they fail.

☐ Bounded repair preserves attempt history and never silently relaxes policy.

☐ Git branch and commit are created only after required pre-commit validation.

☐ Vercel preview corresponds to the exact tested commit.

☐ Preview browser tests and screenshots are available to the reviewer.

☐ Production promotion cannot occur without required role and gate evidence.

☐ Version history links prompt, plan, diff, tests, commit, preview, approval, release, and cost.

☐ Undo/revert and production rollback create new audit history rather than deleting prior history.

☐ Secrets are absent from prompts, logs, artifacts, and repository history.

☐ Cross-tenant authorization tests pass.

☐ Budgets, cancellation, retry, and provider outage behavior are validated.

☐ User can inspect and correct project memory.

☐ Operational alerts and runbooks cover failed release and rollback.

- ☐ All user-facing completion claims are backed by recorded gate evidence.
- ☐ Every LLM invocation passes through the AI Cost Controller and references a routing decision and pricing version.
- ☐ Estimated and actual token/cost records are visible and variance is retained.
- ☐ Hard budget limits prevent further paid operations without policy-authorized action.
- ☐ Model mappings and pricing can be changed without application code deployment.
- ☐ Context optimization demonstrates relevant-file retrieval, cache isolation, and patch-based editing.

Appendix A. Typed Output Examples

These examples define shape, not implementation language. Canonical schemas shall live in the repository and be versioned.

RequirementSpec

{ schemaVersion, changeRequestId, mode, summary, goals[], nonGoals[], assumptions[], acceptanceCriteria[], impactedSurfaces[], constraints[], riskSignals[], attachments[] }

ExecutionPlan

{ schemaVersion, requirementId, baseCommit, riskClass, tasks[{id, objective, expectedFiles[], dependencies[], validations[]}], requestedApprovals[], rollbackConsiderations[], estimatedUsage }

RunResult

{ runId, state, baseCommit, resultCommit, changedFiles[], validations[], previewDeployment, approvals[], warnings[], usage{estimate, actual, variance, pricingVersion, routingDecision, optimizations}, artifacts[], rollbackEligibility }

Appendix B. Open Decisions Before Production

- Select the durable workflow engine and hosting topology.
- Confirm organization-approved AI providers, data terms, retention, and regional controls.
- Choose secrets manager, artifact store, relational database, and observability stack.
- Define supported framework/version matrix and runner images.
- Define environment-specific approval matrix and separation-of-duties rules.
- Set measurable service objectives, retention periods, backup frequency, and restore-test cadence.
- Define production database migration patterns for generated websites.
- Determine whether pull requests are mandatory for single-user projects.
- Select security scanners and dependency/license policy.
- Define model evaluation suite and pilot benchmark repositories.

Appendix C. Final Product Statement

The deliverable is a governed AI software engineering platform whose first plugin builds and modifies websites. Its competitive value is not code generation alone. It is the combination of natural-language control, project-aware context, deterministic engineering gates, cost-aware model orchestration, transparent AI usage, provider-agnostic integrations, visual evidence, immutable version history, controlled release, and reliable recovery. The implementation shall prioritize a trustworthy prompt-to-preview loop first, then expand toward increasingly autonomous and general engineering capabilities without weakening control.